```
# ============================================================
# Smart Campus Energy Consumption Prediction and Efficiency Recommendation
# using Instance-Based Learning and Data Analytics
# ITA0607 - Machine Learning | CO4 + CO5 Integrated Project
# ============================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.kernel_ridge import KernelRidge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

np.random.seed(42)
print('Libraries imported successfully')

# ------------------------------------------------------------
# 1. DATASET GENERATION (>=300 records)
# ------------------------------------------------------------
N = 320
buildings = ['B1_Classrooms', 'B2_Labs', 'B3_Admin', 'B4_Library', 'B5_Hostel']
rows = []
start_date = pd.Timestamp('2025-01-01')

for i in range(N):
    building = np.random.choice(buildings)
    date_time = start_date + pd.Timedelta(hours=int(np.random.uniform(0, 24*90)))

    if building == 'B5_Hostel':
        occupancy = np.random.randint(40, 250)
    elif building == 'B3_Admin':
        occupancy = np.random.randint(5, 60)
    else:
        occupancy = np.random.randint(0, 180)

    temperature = np.round(np.random.normal(29, 4), 1)
    humidity = np.clip(np.round(np.random.normal(60, 10), 1), 20, 95)
    ac_hours = np.clip(np.random.normal(6, 3) + (temperature - 28) * 0.3, 0, 18)
    lighting_hours = np.clip(np.random.normal(8, 2.5), 0, 20)
    equipment_load = np.clip(np.random.normal(35, 15) + occupancy * 0.08, 0, 150)
    renewable = np.clip(np.random.normal(12, 6), 0, 40)

    base = (0.9*occupancy + 3.1*ac_hours + 1.4*lighting_hours + 1.1*equipment_load
            + 0.5*max(temperature-24, 0)*ac_hours*0.4 - 1.2*renewable + 40)
    total_energy = max(base + np.random.normal(0, 15), 5)

    rows.append([building, date_time, occupancy, temperature, humidity, round(ac_hours,2),
                 round(lighting_hours,2), round(equipment_load,2), round(renewable,2), round(total_energy,2)])

df = pd.DataFrame(rows, columns=['Building_ID','Date_Time','Occupancy','Temperature','Humidity',
                                 'AC_Hours','Lighting_Hours','Equipment_Load','Renewable_Energy_kWh','Total_Energy_kWh'])

# Inject missing values and duplicates to demonstrate cleaning
miss_idx = np.random.choice(df.index, 12, replace=False)
for idx in miss_idx:
    col = np.random.choice(['Temperature', 'Humidity', 'Equipment_Load'])
    df.loc[idx, col] = np.nan
df = pd.concat([df, df.sample(6, random_state=1)], ignore_index=True)

df.to_csv('smart_campus_energy.csv', index=False)
print('Dataset shape:', df.shape)
print(df.head())

# ------------------------------------------------------------
# 2. DATA PREPROCESSING (NumPy + Pandas) - CO5
# ------------------------------------------------------------
print(df.info())
print('Missing values per column:\n', df.isnull().sum())

for col in ['Temperature', 'Humidity', 'Equipment_Load']:
    df[col] = df[col].fillna(df[col].median())
print('Missing values after cleaning:\n', df.isnull().sum())
```

```
print('Duplicate rows before removal:', df.duplicated().sum())
df = df.drop_duplicates().reset_index(drop=True)
print('Shape after removing duplicates:', df.shape)

# .loc indexing
print(df.loc[df['Building_ID'] == 'B1_Classrooms', ['Building_ID','Occupancy','Total_Energy_kWh']].head())

# .iloc indexing
print(df.iloc[0:5, 0:5])

# Conditional filtering
high_usage = df[(df['Occupancy'] > 100) & (df['AC_Hours'] > 8)]
print('Rows matching filter (Occupancy>100 and AC_Hours>8):', len(high_usage))

# Descriptive statistics
print(df.describe().round(2))

# NumPy statistics
print('Mean  :', np.mean(df['Total_Energy_kWh']))
print('Median:', np.median(df['Total_Energy_kWh']))
print('Std   :', np.std(df['Total_Energy_kWh']))

# Grouping and aggregation
group_building = df.groupby('Building_ID')['Total_Energy_kWh'].agg(['mean','sum','count']).round(2)
print(group_building)

# Sorting
df_sorted = df.sort_values('Total_Energy_kWh', ascending=False)
print(df_sorted.head())

# File I/O - save cleaned dataset
df.to_csv('cleaned_energy_data.csv', index=False)
print('Cleaned dataset saved as cleaned_energy_data.csv')

# ------------------------------------------------------------
# 3. EXPLORATORY DATA ANALYSIS - CO5
# ------------------------------------------------------------
plt.figure(figsize=(7,5))
group_building['mean'].sort_values().plot(kind='barh', color='#3B82F6')
plt.title('Average Energy Consumption by Building')
plt.xlabel('Average Total Energy (kWh)'); plt.ylabel('Building ID')
plt.tight_layout(); plt.savefig('energy_by_building.png', dpi=120); plt.show()

plt.figure(figsize=(7,5))
plt.scatter(df['Occupancy'], df['Total_Energy_kWh'], alpha=0.5, color='#10B981')
plt.title('Occupancy vs Total Energy Consumption')
plt.xlabel('Occupancy (number of people)'); plt.ylabel('Total Energy (kWh)')
plt.tight_layout(); plt.savefig('occupancy_vs_energy.png', dpi=120); plt.show()

plt.figure(figsize=(7,5))
plt.scatter(df['Temperature'], df['Total_Energy_kWh'], alpha=0.5, color='#F59E0B')
plt.title('Temperature vs Total Energy Consumption')
plt.xlabel('Temperature (°C)'); plt.ylabel('Total Energy (kWh)')
plt.tight_layout(); plt.savefig('temperature_vs_energy.png', dpi=120); plt.show()

plt.figure(figsize=(7,5))
plt.hist(df['Total_Energy_kWh'], bins=25, color='#8B5CF6', edgecolor='white')
plt.title('Distribution of Total Energy Consumption')
plt.xlabel('Total Energy (kWh)'); plt.ylabel('Frequency')
plt.tight_layout(); plt.savefig('energy_distribution.png', dpi=120); plt.show()

# ------------------------------------------------------------
# 4. FEATURE SELECTION, SPLIT, SCALING - CO4
# ------------------------------------------------------------
features = ['Occupancy','Temperature','Humidity','AC_Hours','Lighting_Hours','Equipment_Load','Renewable_Energy_kWh']
target = 'Total_Energy_kWh'

X = df[features].values
y = df[target].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
print('Train size:', X_train_s.shape, ' Test size:', X_test_s.shape)

# ------------------------------------------------------------
```

```
# 5. K-NEAREST NEIGHBOUR REGRESSION - CO4
# ------------------------------------------------------------
knn_results = []
for k in [3, 5, 7, 9]:
    model = KNeighborsRegressor(n_neighbors=k)
    model.fit(X_train_s, y_train)
    pred = model.predict(X_test_s)
    knn_results.append({'K': k,
                        'MAE': round(mean_absolute_error(y_test, pred), 3),
                        'RMSE': round(np.sqrt(mean_squared_error(y_test, pred)), 3),
                        'R2': round(r2_score(y_test, pred), 4)})
knn_df = pd.DataFrame(knn_results)
print(knn_df)
best_knn = knn_df.loc[knn_df['R2'].idxmax()]
print('Best K value:', int(best_knn['K']))

# ------------------------------------------------------------
# 6. LOCALLY WEIGHTED REGRESSION - CO4
# ------------------------------------------------------------
def lwr_predict(X_train, y_train, X_query, tau):
    preds = []
    Xb = np.hstack([np.ones((X_train.shape[0], 1)), X_train])
    for xq in X_query:
        xq_b = np.hstack([[1], xq])
        dists = np.sum((X_train - xq) ** 2, axis=1)
        weights = np.exp(-dists / (2 * tau ** 2))
        W = np.diag(weights)
        XtWX = Xb.T @ W @ Xb
        XtWy = Xb.T @ W @ y_train
        theta = np.linalg.solve(XtWX + 1e-6*np.eye(XtWX.shape[0]), XtWy)
        preds.append(xq_b @ theta)
    return np.array(preds)

lwr_results = []
for tau in [0.5, 1.0, 2.0]:
    pred = lwr_predict(X_train_s, y_train, X_test_s, tau)
    lwr_results.append({'Bandwidth(tau)': tau,
                        'MAE': round(mean_absolute_error(y_test, pred), 3),
                        'RMSE': round(np.sqrt(mean_squared_error(y_test, pred)), 3),
                        'R2': round(r2_score(y_test, pred), 4)})
lwr_df = pd.DataFrame(lwr_results)
print(lwr_df)
best_lwr = lwr_df.loc[lwr_df['R2'].idxmax()]
print('Best bandwidth (tau):', best_lwr['Bandwidth(tau)'])

# ------------------------------------------------------------
# 7. RADIAL BASIS FUNCTION REGRESSION - CO4
# ------------------------------------------------------------
rbf_results = []
for gamma in [0.01, 0.05, 0.1, 0.3]:
    model = KernelRidge(kernel='rbf', gamma=gamma, alpha=1.0)
    model.fit(X_train_s, y_train)
    pred = model.predict(X_test_s)
    rbf_results.append({'Gamma': gamma,
                        'MAE': round(mean_absolute_error(y_test, pred), 3),
                        'RMSE': round(np.sqrt(mean_squared_error(y_test, pred)), 3),
                        'R2': round(r2_score(y_test, pred), 4)})
rbf_df = pd.DataFrame(rbf_results)
print(rbf_df)
best_rbf = rbf_df.loc[rbf_df['R2'].idxmax()]
print('Best gamma:', best_rbf['Gamma'])

# ------------------------------------------------------------
# 8. CASE-BASED LEARNING - CO4
# ------------------------------------------------------------
test_cases = pd.DataFrame([
    {'Occupancy':150,'Temperature':34,'Humidity':55,'AC_Hours':10,'Lighting_Hours':9,'Equipment_Load':80,'Renewable_Energy_kWh':5
    {'Occupancy':20, 'Temperature':26,'Humidity':50,'AC_Hours':2, 'Lighting_Hours':4,'Equipment_Load':15,'Renewable_Energy_kWh':1
    {'Occupancy':200,'Temperature':30,'Humidity':65,'AC_Hours':12,'Lighting_Hours':11,'Equipment_Load':100,'Renewable_Energy_kWh
    {'Occupancy':60, 'Temperature':28,'Humidity':58,'AC_Hours':5, 'Lighting_Hours':6,'Equipment_Load':40,'Renewable_Energy_kWh':1
    {'Occupancy':90, 'Temperature':32,'Humidity':62,'AC_Hours':8, 'Lighting_Hours':7,'Equipment_Load':55,'Renewable_Energy_kWh':1
])

# ------------------------------------------------------------
# 9. ENERGY-EFFICIENCY RECOMMENDATION SYSTEM
# ------------------------------------------------------------
def recommend(row):
```

```
    recs = []
    if row['Occupancy'] > 100:
        recs.append('Use zone-based scheduling to match energy supply with occupancy.')
    if row['AC_Hours'] > 8:
        recs.append('Reduce unnecessary AC usage; use smart temperature control.')
    if row['Lighting_Hours'] > 8:
        recs.append('Install motion-based / daylight-linked lighting controls.')
    if row['Equipment_Load'] > 60:
        recs.append('Schedule high-energy equipment during off-peak hours.')
    if row['Renewable_Energy_kWh'] < 10:
        recs.append('Increase the use of renewable energy sources.')
    if not recs:
        recs.append('Current usage pattern is efficient; maintain existing practices.')
    return ' '.join(recs)

train_scaled_full = scaler.transform(df[features].values)
cbl_rows = []
for tcase_idx, tcase in test_cases.iterrows():
    xq = scaler.transform([tcase.values])[0]
    dists = np.sqrt(np.sum((train_scaled_full - xq) ** 2, axis=1))
    nearest_idx = np.argsort(dists)[:3]
    rec_text = recommend(tcase)
    for rank, idx in enumerate(nearest_idx, start=1):
        cbl_rows.append({'Test_Case': tcase_idx+1, 'Similar_Case_Rank': rank, 'Distance': round(dists[idx],3),
                         'Occupancy': df.loc[idx,'Occupancy'], 'AC_Hours': df.loc[idx,'AC_Hours'],
                         'Lighting_Hours': df.loc[idx,'Lighting_Hours'], 'Equipment_Load': df.loc[idx,'Equipment_Load'],
                         'Historical_Energy_kWh': df.loc[idx,'Total_Energy_kWh'], 'Recommendation': rec_text})
cbl_df = pd.DataFrame(cbl_rows)
print(cbl_df)

# ------------------------------------------------------------
# 10. MODEL EVALUATION AND COMPARISON
# ------------------------------------------------------------
comparison = pd.DataFrame([
    {'Model':'KNN','Parameter':f"K={int(best_knn['K'])}",'MAE':best_knn['MAE'],'RMSE':best_knn['RMSE'],'R2':best_knn['R2']},
    {'Model':'LWR','Parameter':f"tau={best_lwr['Bandwidth(tau)']}",'MAE':best_lwr['MAE'],'RMSE':best_lwr['RMSE'],'R2':best_lwr['
    {'Model':'RBF','Parameter':f"gamma={best_rbf['Gamma']}",'MAE':best_rbf['MAE'],'RMSE':best_rbf['RMSE'],'R2':best_rbf['R2']},
])
print(comparison)
best_overall = comparison.loc[comparison['R2'].idxmax()]
print('Best performing model overall:\n', best_overall)

# ------------------------------------------------------------
# 11. HIGH-ERROR CASE ANALYSIS
# ------------------------------------------------------------
best_k = int(best_knn['K'])
final_knn = KNeighborsRegressor(n_neighbors=best_k)
final_knn.fit(X_train_s, y_train)
pred_final = final_knn.predict(X_test_s)
errors = np.abs(pred_final - y_test)
high_err_idx = np.argsort(errors)[-2:][::-1]

for rank, idx in enumerate(high_err_idx, start=1):
    print(f'High-error case {rank}: Actual={y_test[idx]:.2f}, Predicted={pred_final[idx]:.2f}, '
          f'AbsError={errors[idx]:.2f}, Occupancy={X_test[idx][0]}, Temperature={X_test[idx][1]}, '
          f'AC_Hours={X_test[idx][3]}, Equipment_Load={X_test[idx][5]}')

print('\n✅ Pipeline complete — dataset, EDA plots, KNN/LWR/RBF models, Case-Based Learning, and recommendations all generated.'
```